Mysten Sui Adapter

Security Assessment

October 30th, 2025 — Prepared by OtterSec

Michał Bochnak embe221ed@osec.io

Sangsoo Kang sangsoo@osec.io

Dimitri dshishniashvili@osec.io

Table of Contents

Appendices

01 — Executive Summary

Overview

Mysten engaged OtterSec to assess the `sui-adapter` program. This assessment was conducted between August 11th and October 22nd, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 8 findings throughout this audit engagement.

In particular, we advised utilizing the cached package store function for improved efficiency, as it holds all previously loaded Move packages in memory (OS-MSA-SUG-00). We further advised removing unnecessary resolution table updates (OS-MSA-SUG-02) and redundant shared object validation logic (OS-MSA-SUG-03).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/MystenLabs/sui. This audit was performed on commits from 023c3dc to 953d2ba.

A brief description of the program is as follows:

Name	Description
sui-adapter	The new Sui Adapter statically loads and verifies programmable transactions, performing all type, reference, memory, and drop safety checks before execution. It enforces strict invariants, precomputes package linkage, and uses a borrow-graph model to prevent use-after-move or dangling references.

03 — Findings

Overall, we reported 8 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.

Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	0
INFO	8

04 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MSA-SUG-00	`publish_and_init_package` and `upgrade` redundantly reloads dependency packages from on-chain state via the legacy `fetch_packages` implementation, even though these dependencies were already resolved and cached in memory.
OS-MSA-SUG-01	`new_for_publication` does not update the `versions` field in `ResolvedLinkage`, leaving the new package's version undefined.
OS-MSA-SUG-02	Duplicate updates to `all_versions_resolution_table` and a `get_package` call should be removed, as `add_and_unify` already handles these internally.
OS-MSA-SUG-03	The shared object validation in `context::finish` here is redundant and may be removed.
OS-MSA-SUG-04	The code may be optimized by removing redundant and unnecessary operations to enhance performance.
OS-MSA-SUG-05	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-MSA-SUG-06	Recommendation for updating the codebase to improve functionality and mitigate potential security issues.
OS-MSA-SUG-07	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

Redundant Package Lookup

OS-MSA-SUG-00

Description

In `publish_and_init_package` and `upgrade`, dependencies are fetched via `fetch_packages` from the `linkable_store`, even though they were already resolved during transaction loading.

>_ *sui-adapter/src/static_programmable_transactions/execution/context.rs* RUST

```rust
pub fn publish_and_init_package<Mode: ExecutionMode>([...]) -> Result<ObjectID, ExecutionError>
    ↪ {
    [...]
    let dependencies = self.fetch_packages(dep_ids)?;
    let package = Rc::new(MovePackage::new_initial(
        &modules,
        self.env.protocol_config,
        dependencies.iter().map(|p| p.move_package()),
    )?);
    let package_id = package.id();
    [...]
}
```

`fetch_packages` calls `legacy_ptb::execution::fetch_packages` (as shown below) to reload dependency packages from on-chain state. This results in inefficiency due to the redundant lookup. Instead, dependencies should be retrieved directly from `cached_package_store` to leverage cached `MovePackage` instances.

>_ *sui-adapter/src/static_programmable_transactions/execution/context.rs* RUST

```rust
fn fetch_packages(
    &mut self,
    dependency_ids: &[ObjectID],
) -> Result<Vec<PackageObject>, ExecutionError> {
    legacy_ptb::execution::fetch_packages(&self.env.state_view, dependency_ids)
}
```

Remediation

Utilize `cached_package_store` for improved efficiency, as it holds all previously loaded `MovePackages` in memory.

Patch

Resolved in 12c703c.

Missing Version Mapping Update OS-MSA-SUG-01

Description

`resolved_linkage::new_for_publication` updates ID mappings but does not update the `ResolvedLinkage.versions` field, leaving the new package's version undefined, resulting in incomplete linkage metadata.

>_ *sui-adapter/src/static_programmable_transactions/linkage/resolved_linkage.rs* RUST

```rust
pub fn new_for_publication(
    link_context: ObjectID,
    original_package_id: ObjectID,
    mut resolved_linkage: ResolvedLinkage,
) -> RootedLinkage {
    // original package ID maps to the link context (new package ID) in this context
    resolved_linkage
        .linkage
        .insert(original_package_id, link_context);
    // Add resolution from the new package ID to the original package ID.
    resolved_linkage
        .linkage_resolution
        .insert(link_context, original_package_id);
    let resolved_linkage = Rc::new(resolved_linkage);
    Self {
        link_context: *link_context,
        resolved_linkage,
    }
}
```

Remediation

Ensure to update the `ResolvedLinkage.versions` field in `new_for_publication`.

Patch

Resolved in 13bf451 by removing `ResolvedLinkage.versions`.

Unnecessary Resolution Table Updates OS-MSA-SUG-02

Description

`resolution_table_with_native_packages` and `compute_publication_linkage` (shown below) redundantly update `all_versions_resolution_table`, even though they both call `add_and_unify`, which already performs this update internally. Similarly, there is no requirement to explicitly call `get_package` in `compute_publication_linkage`, as `add_and_unify` already fetches and processes the package.

>_ *sui-adapter/src/static_programmable_transactions/linkage/legacy_linkage.rs* RUST

```rust
// Compute the linkage for a publish or upgrade command. This is a special case because
pub(crate) fn compute_publication_linkage(
    &self,
    deps: &[ObjectID],
    store: &dyn PackageStore,
) -> Result<ResolutionTable, ExecutionError> {
    for id in deps {
        let pkg = get_package(id, store)?;
        add_and_unify(id, store, &mut resolution_table, ConflictResolution::exact)?;
        resolution_table
            .all_versions_resolution_table
            .insert(pkg.id(), pkg.original_package_id());
    }
    Ok(resolution_table)
}
```

Remediation

Remove the updates to `all_versions_resolution_table` in the mentioned functions.

Patch

Resolved in 6ff2075.

Redundant Shared Object Validation OS-MSA-SUG-03

Description

`static_ptb::finish` calls `legacy_ptb::finish` at the end of its execution, rendering the shared object validation loop in `static_ptb::finish` redundant since `legacy_ptb:::finish` already enforces the same checks. Remove this loop to simplify the code and eliminate duplicate execution.

Remediation

Remove the redundant code.

Patch

Resolved in 3b825ac.

Code Optimization OS-MSA-SUG-04

Description

1. In `package_resolution::load_and_verify_packages`, packages do not need to be reloaded and revalidated from the data store if they already exist in the runtime cache, as optimized in `jit_and_cache_package`. This avoids redundant operations and improves performance.

>_ move-vm-runtime/src/runtime/package_resolution.rs RUST

```rust
pub fn load_and_verify_packages(
    vm_config: &VMConfig,
    natives: &NativeFunctions,
    data_store: &impl DataStore,
    allow_loading_failure: bool,
    packages_to_read: &BTreeSet<PackageStorageId>,
) -> VMResult<Vec<verification::ast::Package>> {
    let packages = packages_to_read.iter().cloned().collect::<Vec<_>>();
    let packages = match data_store.load_packages(&packages) {
        Ok(packages) => packages,
        Err(err) if allow_loading_failure => return Err(err),
        Err(err) => { error!("[VM] Error fetching packages {packages_to_read:?}");
            return Err(expect_no_verification_errors(err));
        }
    };
    [...]
}
```

2. In `package_resolution::jit_package_for_publish`, the cache is accessed twice via `cached_package_at`, acquiring the read lock twice unnecessarily. This is inefficient compared to `resolve_packages`, where the cache is accessed only once via `if let Some(...)`. Since `MoveCache` utilizes a single lock for all operations and all access goes through controlled methods (`cached_package_at` and `add_to_cache`), there is no risk of deadlock. Refactoring `jit_package_for_publish` to utilize a single access improves performance.

>_ move-vm-runtime/src/runtime/package_resolution.rs RUST

```rust
pub fn jit_package_for_publish([...]) -> VMResult<Arc<move_cache::Package>> {
    let storage_id = verified_pkg.storage_id;
    if cache.cached_package_at(storage_id).is_some() {
        return Ok(cache.cached_package_at(storage_id).unwrap());
    }
    [...]
}
```

3. The current module loading loop in `translate::package` has `O(n²)` complexity due to repeated dependency checks and re-insertions. Replace it with a proper topological sort to improve performance by resolving module dependencies in a single pass.
4. The tracer currently logs `FreezeRef` as a `POP` and `PUSH`, but since the VM treats it as a no-op with no state change, these logs are unnecessary and may be safely omitted.

Remediation

Include the optimizations listed above.

Missing Validation Logic

OS-MSA-SUG-05

Description

1. `dead_code_elim::blocks` contains an `assert!` that assumes the last instruction of the last basic block is always an unconditional branch. However, this is not guaranteed by Move's `ControlFlowGraph`. If the input bytecode is one basic block itself with no unconditional instruction at the last position, it will just create basic block starting with `ENTRY_BLOCK_ID`. The code should replace the assertion with a conditional check and return a proper invariant violation error if the last basic block's last instruction is not an unconditional branch.

>_ move-vm-runtime/src/jit/optimization/optimizations /dead_code_elim.rs RUST

```rust
fn blocks(
    context: &mut BlockContext<'_, '_, '_>,
    blocks: &mut BTreeMap<ast::Label, Vec<ast::Bytecode>>,
) {
    [...]
    assert!(blocks
        .iter()
        .last()
        .unwrap()
        .1
        .last()
        .unwrap()
        .is_unconditional_branch());
    [...]
}
```

2. The current code in `translate::package` re-derives the `runtime_id` from the first module in the package, even though the `SerializedPackage` already includes a trusted `runtime_id` from the data store. Instead, the function should utilize the provided `runtime_id` and validate that all modules match it.

>_ move-vm-runtime/src/validation/deserialization/translate.rs RUST

```rust
pub(crate) fn package(vm_config: &VMConfig, pkg: SerializedPackage) -> VMResult<Package> {
    [...]
    let runtime_id = *modules.keys().next().expect("non-empty package").address();
    Ok(Package::new(
        runtime_id,
        modules.into_values().collect(),
        pkg,
    ))
}
```

3. The current `pc <= instructions.len` check is incorrect because accessing `instructions[pc]` when `pc == len` results in an out-of-bounds panic. Replacing it with `pc < instructions.len` ensures the program counter always refers to a valid instruction.

>_ move-vm-runtime/src/execution/interpreter/eval.rs RUST

```rust
fn step(
    state: &mut MachineState,
    run_context: &mut RunContext,
    gas_meter: &mut impl GasMeter,
) -> VMResult<StepStatus> {
    let fun_ref = state.call_stack.current_frame.function();
    let instructions = fun_ref.code();
    let pc = state.call_stack.current_frame.pc as usize;
    assert!(
        pc <= instructions.len(),
        "PC beyond instruction count for {}",
        fun_ref.name()
    );
    [...]
}
```

Remediation

Incorporate the missing validation logic.

Code Refactoring OS-MSA-SUG-06

Description

1. `verify_package_no_cyclic_relationships` and `verify_package_valid_linkage` declare `relocation_map` as `HashMap<PackageStorageId, RuntimePackageId>`, while the actul type should be `HashMap<RuntimePackageId, PackageStorageId>` as utilized in `verify_linkage_and_cyclic_checks_for_publication`. While this type mismatch does not create any immediate issues, it is misleading and should be rectified for consistency and to improve code clarity.

>_ move-vm-runtime/src/validation/verification/linkage.rs RUST

```rust
fn verify_package_no_cyclic_relationships(
    package: &[&Module],
    cached_packages: &BTreeMap<PackageStorageId, &Package>,
    relocation_map: &HashMap<PackageStorageId, RuntimePackageId>,
) -> VMResult<()> {[...]}
```

2. The `module_map` variable name in `data_cache::TransactionDataCache` is misleading, as it maps account addresses to full packages, not individual modules. Renaming it to `package_map` will better reflect its purpose and improve code clarity.

>_ move-vm-runtime/src/runtime/data_cache.rs RUST

```rust
pub struct TransactionDataCache<S> {
    pub remote: S,
    module_map: BTreeMap<AccountAddress, AccountDataCache>,
}
```

3. Both `validate_for_publish` and `validate_for_vm_executionuse` in `validation` utilize `RuntimePackageId` where `PackageStorageId` will be more appropriate, as they operate on logical package identities for structural validation rather than runtime-specific instances.
4. The code in `dead_code_elim::eliminate_unreachable` that clears all prior instructions after an unconditional branch is unnecessary because, in Move, such branches always end a basic block.

>_ move-vm-runtime/src/jit/optimization/optimizations/dead_code_elim.rs RUST

```rust
fn eliminate_unreachable(context: &mut BlockContext<'_, '_, '_>, code: &mut
    ↪  Vec<ast::Bytecode>) {
    use ast::Bytecode;
    let mut output_code = vec![];
```

```
    while let Some(instr) = code.pop() {
        [...]
        if instr.is_unconditional_branch() {
            *context.changed = true;
            output_code = vec![];
        }
        output_code.push(instr);
    }[...]
}
```

Remediation

Implement the above-mentioned refactors.

Code Maturity OS-MSA-SUG-07

Description

1. The comment describing the `defining_ids` field in `ast::PackageVirtualTable` and `ast::VMDispatchTables` (shown below) utilizes the word *"mentioned"*, which is misleading, as it implies inclusion of referenced types. In reality, the field only includes package IDs for types that are defined within the package. Replacing *"mentioned"* with *"defined"* improves clarity and prevents confusion about the field's purpose.

>_ move-vm-runtime/src/jit/execution/ast.rs RUST

```rust
#[derive(Debug)]
pub struct VMDispatchTables {
    [...]
    /// Defining ID Set -- a set of all defining IDs on any types mentioned in the
        ↪  package.
    /// [SAFETY] Ordering is not guaranteed
    #[allow(dead_code)]
    pub(crate) defining_id_origins: BTreeMap<PackageStorageId, RuntimePackageId>,
}
```

2. `run` currently frees any leftover call stack frames after execution completes, potentially masking bugs. Since the VM is expected to empty the call stack during execution, leftover frames indicate a logic error. Instead of silently cleaning them up, the function should return an invariant violation error to enforce execution correctness.

>_ move-vm-runtime/src/execution/interpreter/eval.rs RUST

```rust
pub(super) fn run(
    start_state: MachineState,
    vtables: &mut VMDispatchTables,
    vm_config: Arc<VMConfig>,
    extensions: &mut NativeContextExtensions,
    tracer: &mut Option<VMTracer<'_>>,
    gas_meter: &mut impl GasMeter,
) -> VMResult<Vec<Value>> {
    [...]
    heap.free_stack_frame(current_frame.stack_frame)
        .map_err(|e| e.finish(Location::Undefined))?;
    for frame in frames.into_iter().rev() {
        heap.free_stack_frame(frame.stack_frame)
            .map_err(|e| e.finish(Location::Undefined))?;
    }
    Ok(operand_stack.value)
}
```

3. There is a typographical error in the error message in `dispatch_tables::load_type`, where there is a mismatch in the defining ID. The same placeholder `defining_id` is utilized twice, rendering the message uninformative and redundant. The expected ID is from `defining_id`, and the actual is from `datatype.defining_id.address`. Utilize `datatype.defining_id.address` and `defining_id` as the placeholders to correctly show the expected and actual values when a mismatch occurs.

```rust
pub(crate) fn load_type(&self, type_tag: &TypeTag) -> VMResult<Type> {
    Ok(match type_tag {
        [...]
        TypeTag::Struct(struct_tag) => {
           [...]
            // The defining ID should match the defining ID of the datatype that we
            // have loaded.
            if datatype.defining_id.address() != &defining_id {
                return Err(
                    PartialVMError::new(StatusCode::UNKNOWN_INVARIANT_VIOLATION_ERROR)
                        .with_message(format!(
                            "Defining ID {defining_id} does not match defining ID
                                ↪ {defining_id}"
                        ))
                        .finish(Location::Undefined),
                );
            }[...]
        }
    })
}
```

Remediation

Implement the above-mentioned suggestions.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.